SQL Practice Assignment

Database Fundamentals

Instructions

- Answer the following questions using SQL queries.
- Do not use any functions or commands that were not covered in the specific topics provided (e.g., do not use JOINs, Group By, or Wildcards like % in LIKE clauses).
- Ensure your syntax is compatible with SQL Workbench.

Questions

1. Retrieve the complete list of details for every customer stored in the database.
2. Filter the customer list to find only those individuals who reside in the zip code 22801.
3. Create a single column named 'Full Name' that combines the customer's first name, a space, and their last name.
4. Calculate the total cost for each transaction in the customer purchases table by multiplying the quantity purchased by the cost per item. Name this derived column 'Total Spend'.
5. Generate a report of product names that includes a new category label: if the category ID is 1, label it 'Food'; for all other IDs, label it 'Other'.
6. Retrieve a list of full names (first and last concatenated) for customers living in zip code 22801, sorted alphabetically by their last name.
7. Display only the first name and last name for every vendor in the vendor table.
8. specific vendor type 'Fresh Focused'.
9. Extract and display the first 3 characters of the product name for every product in the catalog.
10. Retrieve the original price of items in the vendor inventory, rounded to the nearest whole number (0 decimal places).
11. List customer first names alongside a status label: assign 'Early' if the customer ID is less than 10, and 'Standard' otherwise.
12. Calculate the total cost (quantity multiplied by cost) for all purchases made from Vendor ID 7, rounded to 2 decimal places.
13. List all products in the database, sorting them alphabetically by their product name.
14. Retrieve details for products that belong to either Category ID 1 OR Category ID 2.

15. Display the last 2 characters of the vendor owner's last name for all vendors.
16. Calculate the remainder when the customer ID is divided by 2 to determine if the ID is even or odd.
17. Display booth numbers along with a descriptive label based on price level: 'Prime' for level A, 'Standard' for level B, and 'Economy' for anything else.
18. Select the first 5 products that satisfy two conditions: the category ID is NOT 1 and the product size is NOT 'large'.
19. Show all details from the booth table, ordered by the booth price level in ascending order.
20. Find specific purchase records where the quantity purchased is exactly 1 AND the vendor ID is 7.
21. Determine the length of the string in the product name column for every product.
22. Create a derived column that calculates the square (power of 2) of the quantity field in the customer purchases table.
23. Categorize the vendor inventory based on quantity: label records with a quantity greater than 20 as 'High Stock', and all others as 'Low Stock'.
24. Display the product name and the character count of the name, but only for products where the name is longer than 10 characters.
25. Retrieve the first 10 records from the customer purchases table to get a sample of the data.
26. Find and list all products where the product category ID is NOT equal to 5.
27. Create a unique identifier for vendors by concatenating the vendor type and vendor name, separated by a hyphen.
28. Calculate a 'Sale Price' for all inventory items by dividing the original price by 2.
29. Create a size classification for products: if the product size is 'large', display 'L', otherwise display 'S/M'.
30. List inventory items where the total calculated value (quantity multiplied by price) exceeds 50.
31. Display the product ID and product size for all items, sorted by their category ID in descending order.
32. List details for booths that have a price level of either 'A' or 'B'.
33. extract the first 3 digits of the customer zip code to analyze the region prefix.
34. Find the remainder when the product ID is divided by 5.
35. Label purchase quantities: if the quantity is between 1 and 5 (inclusive), label it 'Small Batch'; otherwise, label it 'Bulk'.
36. Display the booth description and price level for booths with descriptions longer than 20 characters, sorted by booth number.
37. Select a subset of vendors: retrieve 5 vendor records, but skip the first 2 records in the table.

38. Filter the inventory table to find records where the quantity on hand is between 10 and 20.
39. Display booth descriptions with all leading and trailing whitespace removed.
40. Calculate the projected revenue for inventory items (quantity * original price) and round the result to 1 decimal place.
41. Create a readable label for vendor types: if the type is 'Prepared Foods', display 'Ready to Eat'; otherwise, display 'Ingredients'.
42. Retrieve records for customers whose ID is between 1 and 20 AND who reside in zip code 22821.
43. List all customer zip codes, sorted first by zip code in ascending order, and then by customer last name in descending order.
44. Find customers with a customer ID number between 5 and 15.
45. Generate a simple code for each customer by combining the first letter of their first name and the last letter of their last name.
46. Calculate the cube (power of 3) of the quantity field in the customer purchases table.
47. Create a regional label based on zip code: 'Central' for 22801, 'East' for 22802, and 'External' for all others.
48. Create a masked version of vendor names that shows only the first 3 letters followed by '...'.
49. Display the booth assignments table, but limit the output to only 7 records.
50. Find products where the category is NOT 3 AND the product size is 'medium'.
51. Calculate the character length of the booth description field.
52. Generate a 'New ID' for products by adding the number 10 to the existing product ID.
53. Convert the booth type into a numerical flag: return 1 if the type is 'Large', and 0 for any other type.
54. Find purchases where the cost per quantity is greater than 5 OR the quantity is greater than 10, limited to the top 15 results.
55. Show the inventory data sorted by quantity in descending order, and then by original price in ascending order.
56. Find booth assignments for Vendor 7 where the booth number is either 11 or 12.
57. Create a 'Location Code' by combining the booth price level and the booth type.
58. Calculate the difference between the 'cost to customer per qty' and the number 5 for every purchase record.
59. Classify purchase costs: label as 'Expensive' if cost > 10, 'Moderate' if between 5 and 10, and 'Cheap' otherwise.
60. Calculate the modulo 3 of the vendor ID and display only those records where the result is not 0.

Answer Key

1. ```
   SELECT * FROM customer;
   ```

2. ```
   SELECT * FROM customer WHERE customer_zip = 22801;
   ```

3. ```
   SELECT CONCAT(customer_first_name, ' ', customer_last_name) AS full_name
   FROM customer;
   ```

4. ```
   SELECT quantity * cost_to_customer_per_qty AS total_spend
   FROM customer_purchases;
   ```

5. ```
   SELECT product_name,
   CASE
       WHEN product_category_id = 1 THEN 'Food'
       ELSE 'Other'
   END AS category_label
   FROM product;
   ```

6. ```
   SELECT CONCAT(customer_first_name, ' ', customer_last_name)
   FROM customer
   WHERE customer_zip = 22801
   ORDER BY customer_last_name;
   ```

7. ```
   SELECT vendor_owner_first_name, vendor_owner_last_name FROM vendor;
   ```

8. ```
   SELECT * FROM vendor WHERE vendor_type = 'Fresh Focused';
   ```

9. ```
   SELECT LEFT(product_name, 3) FROM product;
   ```

10. ```
    SELECT ROUND(original_price, 0) FROM vendor_inventory;
    ```

11. ```
    SELECT customer_first_name,
    CASE
        WHEN customer_id < 10 THEN 'Early'
        ELSE 'Standard'
    END AS status
    FROM customer;
    ```

12. ```
    SELECT ROUND(quantity * cost_to_customer_per_qty, 2)
    FROM customer_purchases
    WHERE vendor_id = 7;
    ```

13. ```
    SELECT * FROM product ORDER BY product_name;
    ```

14. ```
    SELECT * FROM product
    WHERE product_category_id = 1 OR product_category_id = 2;
    ```

15. ```
    SELECT RIGHT(vendor_owner_last_name, 2) FROM vendor;
    ```

16. ```
    SELECT MOD(customer_id, 2) FROM customer;
    ```

17. ```
    SELECT booth_number,
    CASE
        WHEN booth_price_level = 'A' THEN 'Prime'
        WHEN booth_price_level = 'B' THEN 'Standard'
        ELSE 'Economy'
    END AS label
    FROM booth;
    ```

18. ```
    SELECT * FROM product
    WHERE product_category_id <> 1 AND product_size <> 'large'
    LIMIT 5;
    ```

19. ```
    SELECT * FROM booth ORDER BY booth_price_level ASC;
    ```

20. ```
    SELECT * FROM customer_purchases
    WHERE quantity = 1 AND vendor_id = 7;
    ```

21. ```
    SELECT LENGTH(product_name) FROM product;
    ```

22. ```
    SELECT POW(quantity, 2) FROM customer_purchases;
    ```

23. ```
    SELECT quantity,
    CASE
        WHEN quantity > 20 THEN 'High Stock'
        ELSE 'Low Stock'
    END AS stock_level
    FROM vendor_inventory;
    ```

24. ```
    SELECT product_name, LENGTH(product_name)
    FROM product
    WHERE LENGTH(product_name) > 10;
    ```

25. ```
    SELECT * FROM customer_purchases LIMIT 10;
    ```
    ```

26. `SELECT * FROM product WHERE product_category_id <> 5;`

27. `SELECT CONCAT(vendor_type, '-', vendor_name) FROM vendor;`

28. `SELECT original_price / 2 AS sale_price FROM vendor_inventory;`

29.
```
SELECT product_name, product_size,
CASE
    WHEN product_size = 'large' THEN 'L'
    ELSE 'S/M'
END AS size_class
FROM product;
```

30.
```
SELECT *, quantity * original_price AS total_value
FROM vendor_inventory
WHERE (quantity * original_price) > 50;
```

31.
```
SELECT product_id, product_size
FROM product
ORDER BY product_category_id DESC;
```

32.
```
SELECT * FROM booth
WHERE booth_price_level = 'A' OR booth_price_level = 'B';
```

33. `SELECT LEFT(customer_zip, 3) FROM customer;`

34. `SELECT product_id % 5 FROM product;`

35.
```
SELECT quantity,
CASE
    WHEN quantity BETWEEN 1 AND 5 THEN 'Small Batch'
    ELSE 'Bulk'
END AS batch_type
FROM customer_purchases;
```

36.
```
SELECT booth_description, booth_price_level
FROM booth
WHERE LENGTH(booth_description) > 20
ORDER BY booth_number;
```

37. `SELECT * FROM vendor LIMIT 5 OFFSET 2;`

38. `SELECT * FROM vendor_inventory WHERE quantity BETWEEN 10 AND 20;`

39. ```
    SELECT TRIM(booth_description) FROM booth;
    ```

40. ```
    SELECT ROUND(quantity * original_price, 1) FROM vendor_inventory;
    ```

41. ```
    SELECT vendor_name,
    CASE
        WHEN vendor_type = 'Prepared Foods' THEN 'Ready to Eat'
        ELSE 'Ingredients'
    END AS type_label
    FROM vendor;
    ```

42. ```
    SELECT * FROM customer
    WHERE customer_id BETWEEN 1 AND 20 AND customer_zip = 22821;
    ```

43. ```
    SELECT customer_zip, customer_last_name
    FROM customer
    ORDER BY customer_zip ASC, customer_last_name DESC;
    ```

44. ```
    SELECT * FROM customer WHERE customer_id BETWEEN 5 AND 15;
    ```

45. ```
    SELECT CONCAT(LEFT(customer_first_name, 1), RIGHT(customer_last_name, 1))
    FROM customer;
    ```

46. ```
    SELECT POW(quantity, 3) FROM customer_purchases;
    ```

47. ```
    SELECT customer_zip,
    CASE
        WHEN customer_zip = 22801 THEN 'Central'
        WHEN customer_zip = 22802 THEN 'East'
        ELSE 'External'
    END AS zip_region
    FROM customer;
    ```

48. ```
    SELECT CONCAT(LEFT(vendor_name, 3), '...') AS masked_name
    FROM vendor;
    ```

49. ```
    SELECT * FROM vendor_booth_assignments LIMIT 7;
    ```

50. ```
    SELECT * FROM product
    WHERE product_category_id != 3 AND product_size = 'medium';
    ```

51. ```
    SELECT LENGTH(booth_description) FROM booth;
    ```
    ```

52.
```
SELECT product_id, product_id + 10 AS new_id FROM product;
```

53.
```
SELECT booth_type,
CASE
    WHEN booth_type = 'Large' THEN 1
    ELSE 0
END AS is_large
FROM booth;
```

54.
```
SELECT * FROM customer_purchases
WHERE cost_to_customer_per_qty > 5 OR quantity > 10
LIMIT 15;
```

55.
```
SELECT * FROM vendor_inventory
ORDER BY quantity DESC, original_price ASC;
```

56.
```
SELECT * FROM vendor_booth_assignments
WHERE vendor_id = 7 AND (booth_number = 11 OR booth_number = 12);
```

57.
```
SELECT CONCAT(booth_price_level, booth_type) AS location_code
FROM booth;
```

58.
```
SELECT cost_to_customer_per_qty - 5 FROM customer_purchases;
```

59.
```
SELECT cost_to_customer_per_qty,
CASE
    WHEN cost_to_customer_per_qty > 10 THEN 'Expensive'
    WHEN cost_to_customer_per_qty BETWEEN 5 AND 10 THEN 'Moderate'
    ELSE 'Cheap'
END AS price_label
FROM customer_purchases;
```

60.
```
SELECT *, MOD(vendor_id, 3)
FROM vendor
WHERE MOD(vendor_id, 3) != 0;
```